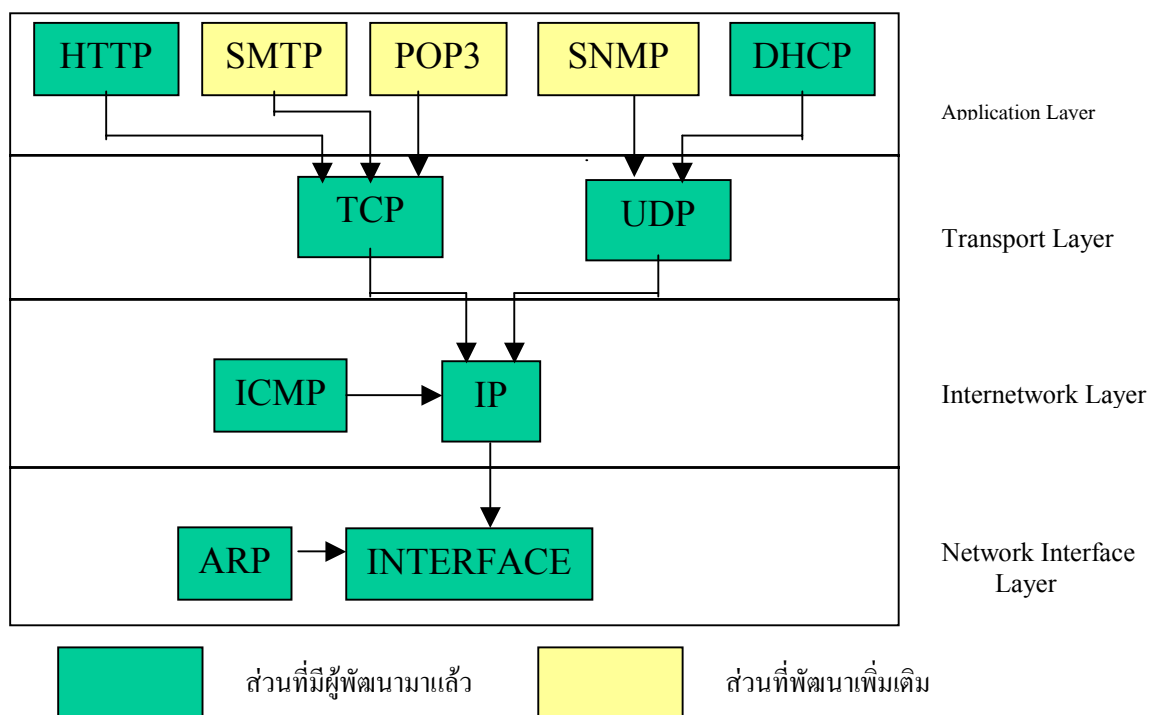


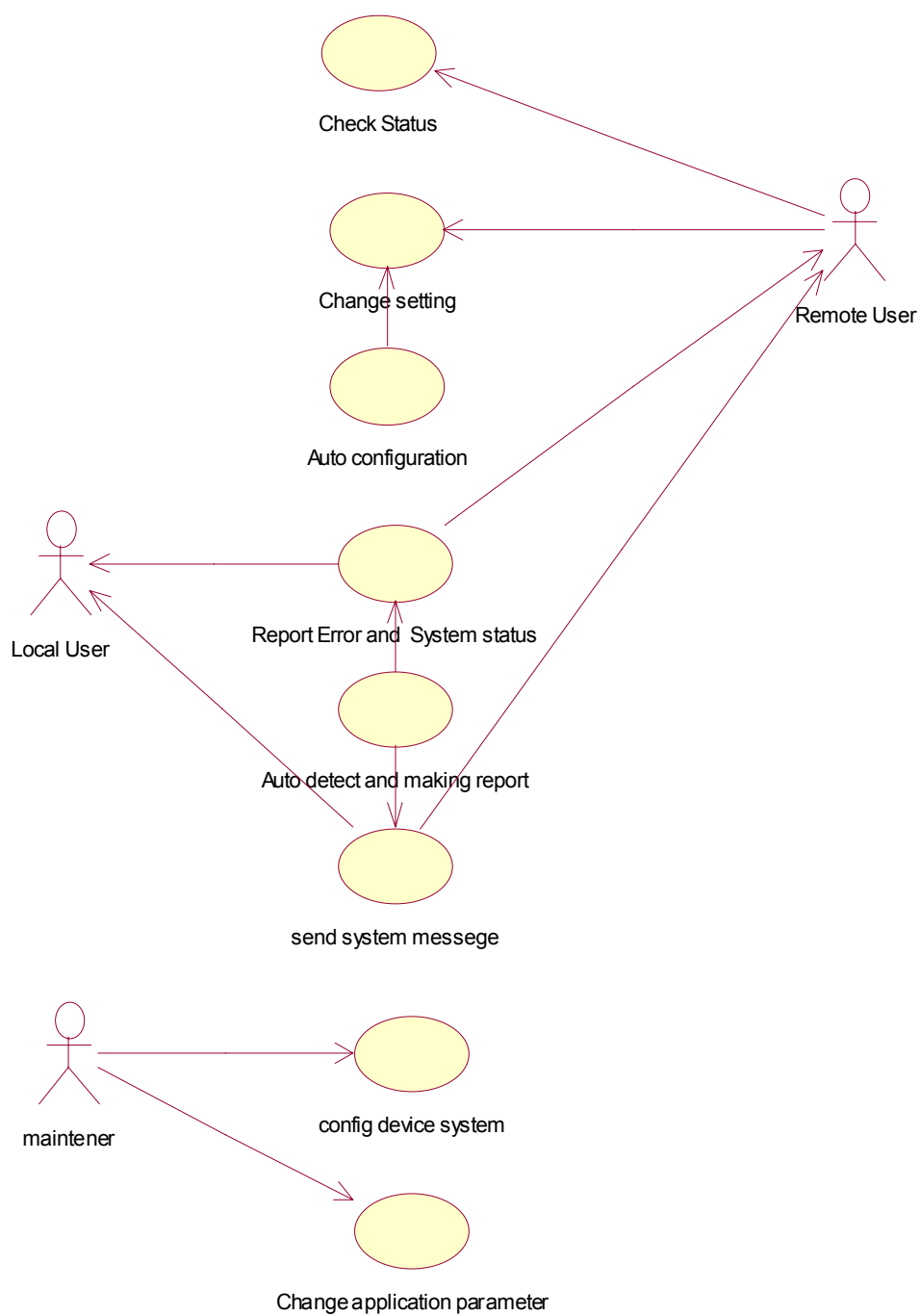
บทที่ 5

แนวทางการออกแบบซอฟต์แวร์

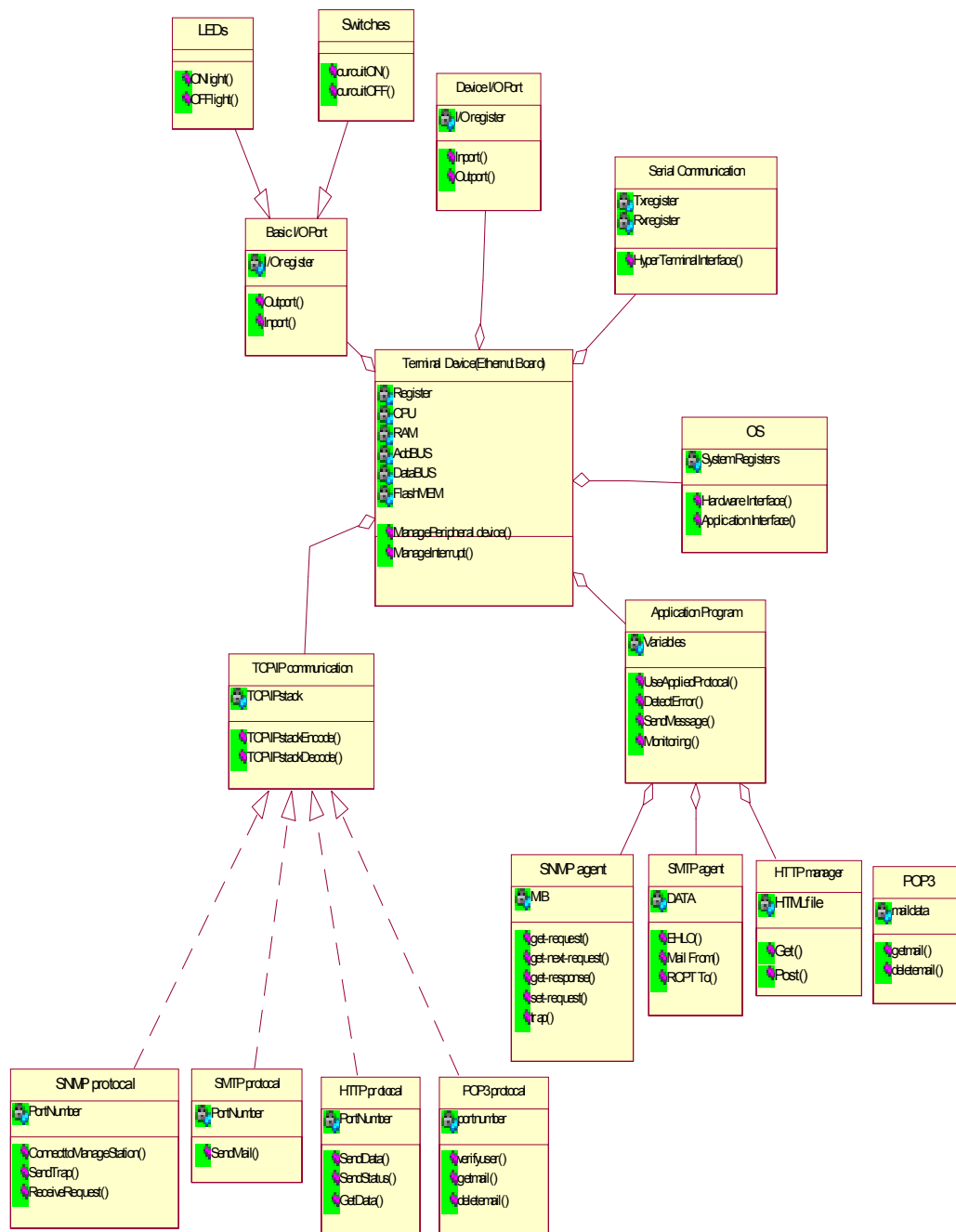
แนวทางการพัฒนานั้นทางผู้จัดทำได้ใช้แนวทางของ Nut/OS และ Nut/Net โดยได้ทำการเพิ่มส่วนที่เป็นโปรโตคอลที่ยังไม่มีการพัฒนา โดยได้ทำการพัฒนาโปรโตคอลเพิ่มอีก 3 โปรโตคอลคือ SNMP, SMTP, POP3 ดังที่แสดงไว้ในรูป



รูปที่ 5-1 แสดงโปรโตคอลส่วนที่ทำการพัฒนาเพิ่มเติม



รูปที่ 5-2 แสดงความสัมพันธ์ระหว่างผู้ใช้งานกับการใช้งานระบบ



รูปที่ 5-3 แสดงส่วนประกอบทั้งทางด้านฮาร์ดแวร์และซอฟต์แวร์ของระบบ โดยแต่ละ โปรโตคอลที่ได้ทำการพัฒนานั้นมีจุดประสงค์และหลักการออกแบบดังมีรายละเอียดดังต่อไปนี้

POP3 SERVER

จุดประสงค์การใช้งานโปรโตคอล POP3

เนื่องจากอุปกรณ์ Embedded มักจะมีข้อจำกัดในการใช้งานหน่วยความจำที่จำกัดหรือการบันทึกค่าต่างๆไม่สามารถทำได้ เช่น ไม่มี flash memory ดังนั้นการอ่านข้อมูลจากตัวอุปกรณ์และทำการบันทึกค่าต่างๆไว้จึงเป็นการเพิ่มประสิทธิภาพให้กับอุปกรณ์ Embedded เป็นอย่างมาก

โปรโตคอล POP3 เป็นการอ่านเมลที่อยู่บนฝั่งเซิร์ฟเวอร์มากเก็บไว้ยังฝั่งไคลเอ็นต์ ซึ่งเราสามารถนำมาประยุกต์ใช้งานได้ตามลักษณะงานที่เราต้องการได้ โดยให้เมลที่เก็บไว้ยังฝั่งเซิร์ฟเวอร์เป็นข้อมูลหรือรายงานที่เราต้องการทราบ และเมื่อต้องการอ่านข้อมูลก็ทำการติดต่อจากฝั่งไคลเอ็นต์ไปยังฝั่งเซิร์ฟเวอร์เพื่ออ่านข้อมูลหรือรายงานนั้นๆมาได้หลังจากนั้นจึงปิดช่องทางการสื่อสาร ดังนั้นการใช้งานโปรโตคอล POP3 จึงไม่จำเป็นต้องเปิดช่องทางการสื่อสารไว้ตลอดเวลา

หลักการออกแบบ

1. เนื่องจากการใช้งานโปรโตคอล POP3 นั้นเป็นการดึงข้อมูลจากตัวบอร์ดโดยตรง จึงต้องมีการยืนยันตัวบุคคลที่มีสิทธิ์ที่จะใช้งาน ซึ่งจะใช้ขั้นตอนของการเช็ค Username และ Password
2. การทำงานจะโดยอ้างอิงมาตรฐานชุดคำสั่งต่ำสุดที่ระบุไว้เพื่อการใช้งานที่พื้นฐาน
3. การทำงานของโปรแกรมจะแสดงขนาดของหน่วยความจำที่เหลือขณะที่ผู้ใช้ทำการสอบถาม ซึ่งจะส่งมาในลักษณะของข้อความ

ฟังก์ชันการทำงาน

รวบรวม mail (ที่เก็บสถิติการทำงาน) ที่จะทำการส่งไว้ที่ Terminal Device เพื่อบอกสถานะของ Terminal Device โดยฝ่ายผู้ควบคุม สามารถที่จะดึง mail จาก Terminal Device ได้โดยใช้ Program อ่าน mail เช่น Eudora โดยผ่านทางระบบ Network เพื่อดูการทำงานของ Terminal Device ในระหว่างที่ไม่มีการเชื่อมต่อกับ Network

สิ่งที่ต้องทดสอบ

- สามารถ connect เข้าสู่ POP3 Server ได้หรือไม่
- สามารถดึง mail จาก POP3 Server ได้หรือไม่

ขั้นตอนการทดสอบ

- ทำการสื่อสารโดยใช้ Telnet กับ POP3 Server ที่ให้บริการอยู่ทั่วไปเพื่อดูการใช้ Command ต่างๆของ POP3
- ทำการจับ Packet โดยใช้ Program Ethereal เพื่อดูการส่ง Packet ระหว่าง POP3 Client กับ

POP3 Server ที่สร้างขึ้นว่ามีขั้นตอนอย่างไร เพื่อความีขั้นตอนไหนที่ผิดพลาด

- ทำการ Connect เข้าสู่ POP3 Server on Terminal Device โดยใช้ Program Eudora แล้วทำการดึง mail จาก Terminal Device แล้วดูว่า mail ที่ได้รับมา มีความถูกต้องหรือไม่
- ลองทำการติดต่อโดยใช้ Program อื่นๆ เช่น Outlook Express

ผลการทดสอบ

- จากการทดลองสามารถที่จะติดต่อกับ POP3 Server ได้ และสามารถดึง mail เข้ามาได้ และมีความถูกต้องของ เนื้อหาของ mail แต่สามารถใช้ได้กับ Program Eudora เท่านั้น ส่วน Outlook Express ไม่สามารถทำงานได้ เพราะขั้นตอนการติดต่อเมื่อทำการ Authentication ได้แล้ว ฟังก์ชัน POP3 Client (Outlook Express) ไม่ทำการส่งคำสั่ง STAT ทำให้การติดต่อไม่สามารถดำเนินการต่อไปได้

ข้อจำกัดของการทำงาน

1. เพื่อความปลอดภัยในการใช้งานจึงต้องมีการระบุ Username กับ Password โดยหากไม่สามารถระบุถึงผู้ใช้งานได้ก็จะไม่สามารถใช้งานคำสั่งใดๆได้
2. การยืนยันบุคคลจะเป็นการส่ง Username กับ Password มาใช้กับตัวบอร์คโดยไม่มีการเข้ารหัสแต่อย่างใด
3. เนื่องจากข้อจำกัดของขนาดหน่วยความจำทำให้ไม่สามารถจะทำการส่งข้อความที่มีขนาดใหญ่ หรือไฟล์ได้

แนวทางในการใช้งานและพัฒนาต่อเนื่อง

1. ใช้ในการอ่านค่าต่างๆ ที่จำเป็นต้องบันทึกไว้เพื่อทำเป็นฐานข้อมูล หรือเก็บไว้เป็นรายงานการทำงานของบอร์คในแต่ละช่วงเวลาที่ต้องการ
2. สามารถเขียนโปรแกรมให้ความสามารถที่หลากหลายขึ้น เช่น มีรายนามผู้ใช้งานหลายคน (multiuser)

POP3 Commands

คำสั่ง	พารามิเตอร์	สถานะ	รายละเอียด	คำสั่งที่ใช้
USER ¹	ชื่อผู้ใช้งาน	ขออนุมัติ	แจ้งชื่อผู้ใช้ และระบุเมลบ็อกซ์ที่จะใช้	ใช้
PASS ¹	Password	ขออนุมัติ	เป็นคำสั่งที่ระบุ Password โดยจะใช้คำสั่งต่อจาก USER	ใช้
APOP ²	ชื่อ,Password	ขออนุมัติ	ทำหน้าที่เหมือนคำสั่ง USER และ PASS รวมกันแต่ข้อมูลจะถูกเข้ารหัสก่อนส่งกลับไป	ไม่ใช่
STAT ¹	ไม่ระบุ	รับส่งรายการ	เป็นคำสั่งตรวจสอบสภาพ Server	ใช้
UIDL ²	หมายเลขข้อความ	รับส่งรายการ	ใช้ตรวจสอบหมายเลขประจำของ E-mail	ใช้
LIST ¹	หมายเลขข้อความ	รับส่งรายการ	ใช้ตรวจสอบหมายเลขและขนาดของ E-mail	ใช้
RETR ¹	ข้อความ	รับส่งรายการ	เป็นคำสั่งที่ใช้ส่งข้อมูลของEmail	ใช้
DELE ¹	ข้อความ	รับส่งรายการ	เป็นการระบุเครื่องหมายการลบลงใน E-mail ที่จะลบ และE-mail เหล่านั้นจะถูกลบออกจากเมลบ็อกซ์เมื่อใช้คำสั่ง QUIT เมื่อสิ้นสุดการทำงาน	ใช้
RSET ¹	ไม่ระบุ	รับส่งรายการ	คำสั่งนี้จะยกเลิกเครื่องหมายการลบE-mail ที่เคยกำหนดไว้ด้วยคำสั่ง DELE ออกไปทุกๆE-mail	ไม่ใช่
TOP ²	หมายเลขข้อความจำนวนบรรทัด	รับส่งรายการ	Server จะส่งข้อมูลย้อนกลับไปเท่ากับจำนวนบรรทัดที่ระบุ	ไม่ใช่
NOOP ¹	ไม่ระบุ	รับส่งรายการ	เป็นคำสั่ง No Operation	ไม่ใช่

QUIT'	ไม่ระบุ	รับส่งรายการและขออนุมัติ	ใช้เมื่อจบการทำงาน หากมีE-mail ซึ่งทำเครื่องหมายว่าจะลบไว้E-mail เหล่านั้นจะถูกลบออกจาก เมล์บ็อกซ์ในขั้นตอนนี้นี้	ใช้
-------	---------	--------------------------	---	-----

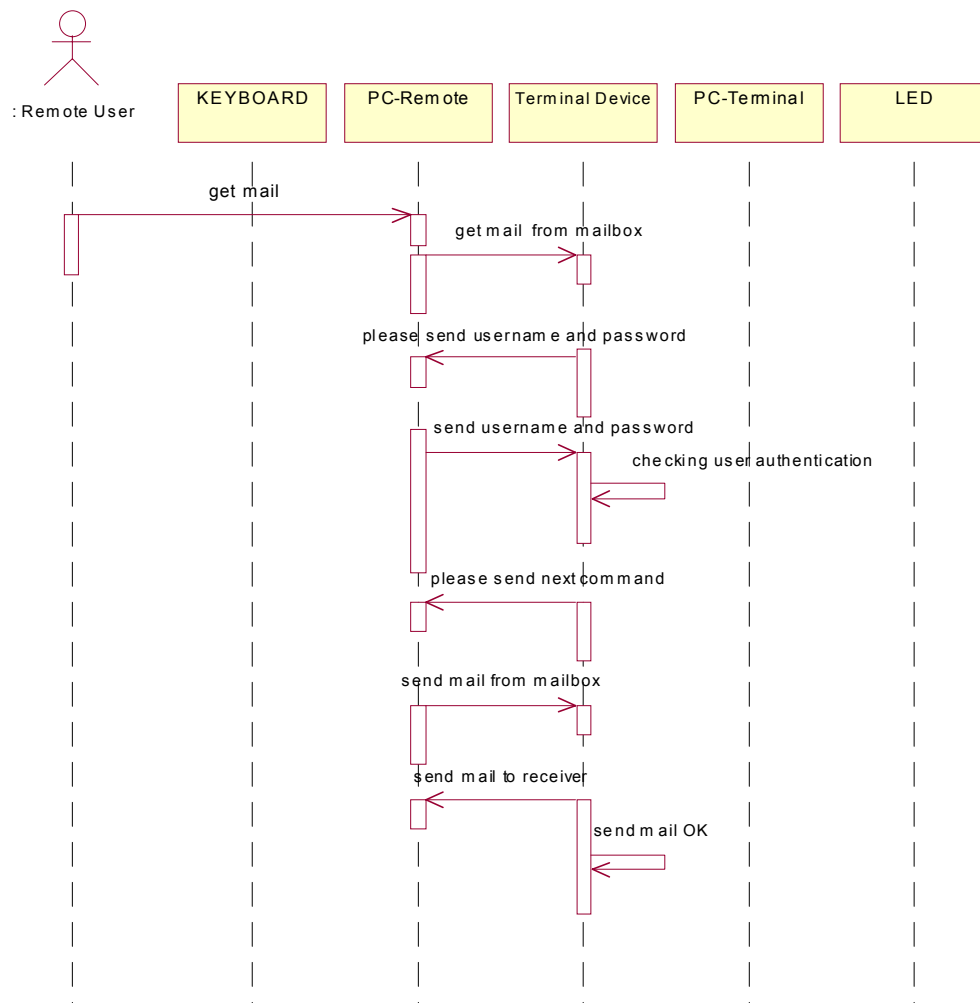
หมายเหตุ

1 Minimal POP3 Commands

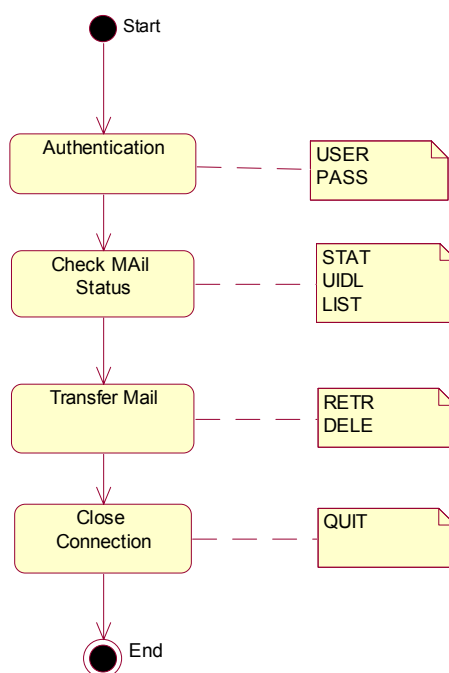
2 Optional POP3 Commands

POP3 Replies : + OK , -ERR

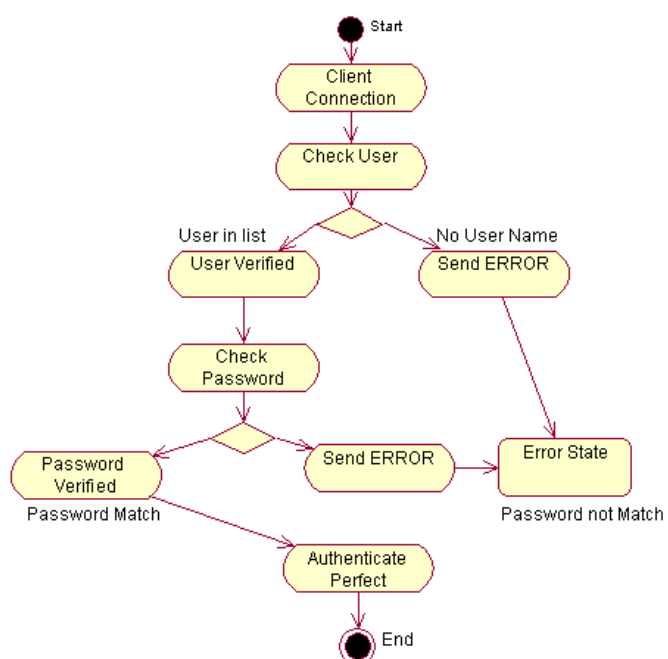
ตารางที่ 5-1 แสดง POP3 Commands ที่ได้เลือกใช้ในวิทยานิพนธ์



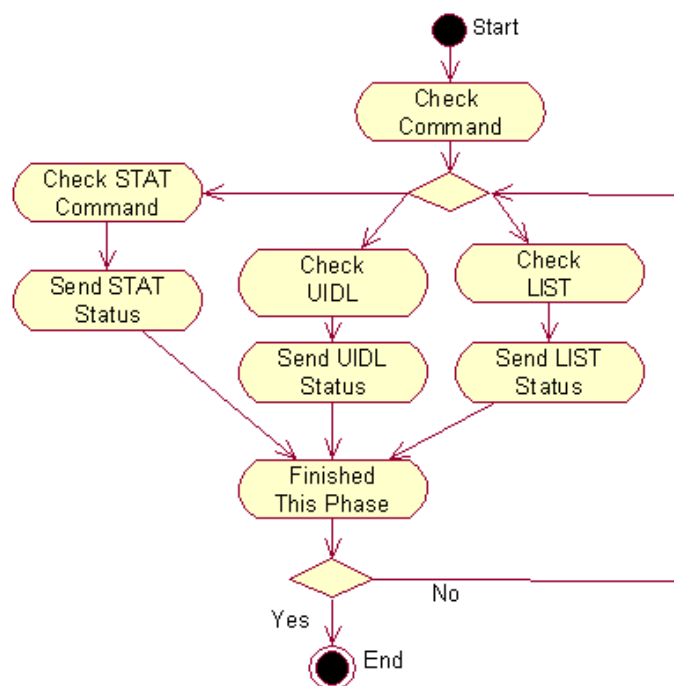
รูปที่ 5-4 แสดง Sequence Diagram ของการใช้งาน POP3 server



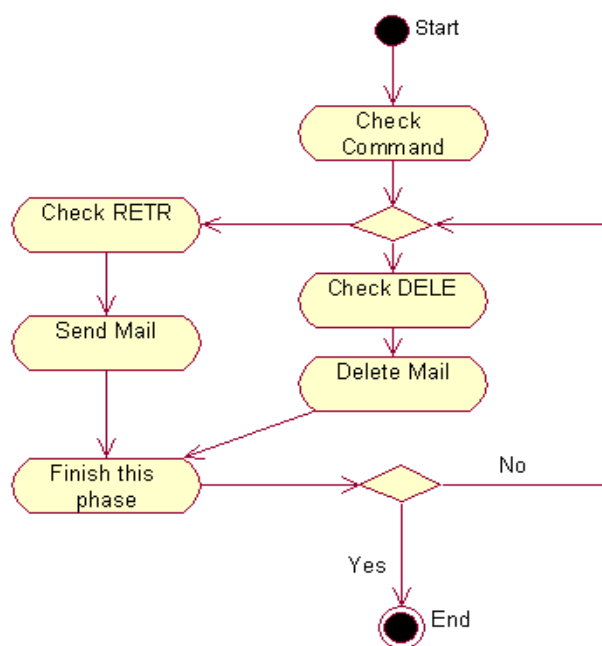
รูปที่ 5-5 แสดงสถานะการทำงานโปรโตคอล POP3



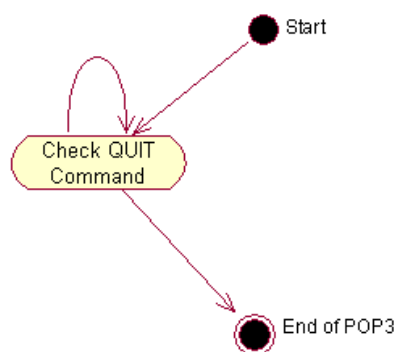
รูปที่ 5-6 แสดงขั้นตอนการพิสูจน์ผู้ใช้งานของโปรโตคอล POP3



รูปที่ 5-7 แสดงขั้นตอนการตรวจสอบสถานะของจดหมายที่มีอยู่



รูปที่ 5-8 แสดงขั้นตอนการส่งเนื้อหาของจดหมาย



รูปที่ 5-9 แสดงขั้นตอนของการจบการทำงาน

SMTP

จุดประสงค์การใช้งานโปรโตคอล SMTP

ลักษณะการใช้งานที่เหมาะสมกับโปรโตคอล SMTP จะเป็นลักษณะงานที่ไม่ได้มีการจองช่องทางการสื่อสารไว้ตลอดเวลา โดยจะทำการติดต่อเมื่อต้องการจะรับส่งข้อมูลที่ต้องการ และปิดช่องทางการสื่อสารเมื่อมีการรับส่งข้อมูลระหว่างกันเรียบร้อยแล้ว เช่น เมื่อบอร์ดทำการตรวจวัดรอบการทำงานของมอเตอร์และบันทึกความเร็วของรอบไว้เป็นรายงานขนาดเล็ก เมื่อถึงเวลาที่กำหนดบอร์ดจะทำการติดต่อไปยังเมลเซิร์ฟเวอร์และทำการส่งรายงานหลังจากนั้นจึงปิดช่องทางการสื่อสาร

หลักการออกแบบ

- 1.การทำงานจะแบบออกเป็นสองแบบ คือ เมลเซิร์ฟเวอร์ทำหน้าที่รับเมลเพื่อนำมาประมวลผลกับ เมลเอเจนต์ที่ทำหน้าที่ส่งเมลไปยังเมลเซิร์ฟเวอร์อื่นๆ
- 2.การทำงานทั้งสองแบบนี้ อาศัยหลักการของการรับส่งข้อมูลเช่นเดียวกัน โดยอ้างอิงกับมาตรฐานชุดคำสั่งต่ำสุดที่ระบุไว้เพื่อการใช้งานที่พื้นฐาน
- 3.เนื่องจากตัวบอร์ดเป็นอุปกรณ์ขนาดเล็กและมีความสามารถที่จำกัดในการใช้งานบางอย่าง จึงอาจยังไม่สามารถกระทำได้ เช่น การรับส่งไฟล์ การเข้ารหัสและถอดรหัสโดยได้มีการแบ่งการทำงานเป็น 2 ส่วน คือ ฟังก์ชันรับ และฟังก์ชันส่ง

SMTP ฟังก์ชัน (SMTP Server)

ฟังก์ชันการทำงาน

สามารถรับเมลจากที่ต่างๆ เพื่อนำมาวิเคราะห์หรือสั่งงานที่ตัวบอร์ดได้ โดยการ telnet หรือส่งจาก ที่ต่างๆ ได้ แต่ไม่สามารถรับเมลเข้ามาที่ตัวบอร์ดแล้วเก็บเอาไว้อ้างอิงได้ โดยมีคำสั่งการใช้งาน ดังนี้

EHLO	-คำสั่งตรวจสอบว่า server ให้บริการ smtp ส่วนขยายอะไรบ้าง
VERFY	-คำสั่งตรวจสอบ ADDRESS ของผู้รับเมล (ใช้ตรวจสอบว่ามี address ปลายทางหรือไม่)
MAIL	-คำสั่งบอกaddress ปลายทางที่ต้องการส่งไป
RCPT	-คำสั่งบอกaddress ปลายทางที่ต้องการส่งไป
DATA	-คำสั่งใช้บอกว่าทางส่งจะเริ่มการส่งข้อความ
QUIT	-คำสั่งขอยกเลิกการติดต่อ

สิ่งที่ต้องทดสอบ

- สามารถรับการ Connect จากที่ต่างๆ ได้ ทาง port 25
- สามารถรับเมลจากที่ต่างๆ เข้าสู่ Smtп Server ได้ทาง telnet port 25
- สามารถรับเมลจากที่ต่างๆ เข้าสู่ Smtп Server ได้ทางการส่งเมลจาก web ทั่วไปได้
- สามารถรับเมลจากที่ต่างๆ เข้าสู่ Smtп Server ได้ทาง Program ที่ใช้ส่งเมล ทั่วไปได้

ขั้นตอนการทดสอบ

- ทำการรับเมลล์จาก การ telnet แล้วมาแสดงผลทางหน้าจอ Hyperterminal
- ทำการรับเมลล์จาก การ web ต่างๆ แล้วมาแสดงผลทางหน้าจอ Hyperterminal
- ทำการรับเมลล์จาก การ Program สำหรับส่งเมลล์ แล้วมาแสดงผลทางหน้าจอ Hyperterminal

ผลการทดสอบ

จากการทดลองสามารถรับเมลล์จากการ telnet เข้า port 25 หรือ จากการส่งตามหน้า web ของ www.yahoo.com แล้วนำมาแสดงผลที่หน้าจอ Hyperterminal ได้

SMTP ฟังก์ชัน(Smtp Client)

ฟังก์ชันการทำงาน

เพื่อสามารถส่งเมลล์จากตัวบอร์ด Terminal Devices เพื่อรายงานผล หรือ สภาพต่างๆของตัวบอร์ดกลับมายังฝั่งผู้ตรวจสอบ เพื่อสามารถเปลี่ยนแปลงและแก้ไขได้ โดยเราไม่ต้องมีการเชื่อมต่อตัวบอร์ดกับระบบตลอดเวลาเพื่อลดภาระของระบบ

ทางคณะผู้จัดทำได้เขียนฟังก์ชันการทำงานขึ้นมาเพื่อให้สามารถเรียกใช้งานได้สะดวก โดยมีฟังก์ชันที่สามารถเรียกใช้ได้คือ

```
u_long IPresolve(u_char *servername);
```

-ใช้สำหรับหาIPของเมลล์เซฟเวอร์

```
int MailConnect(TCPSOCKET *sock,u_long mailip,int port);
```

-ใช้สำหรับติดต่อไปยังเมลล์เซฟเวอร์ที่ให้บริการSMTPเพื่อเปิดช่องทาง การติดต่อ

```
int GetMailEhlo(TCPSOCKET *sock,u_char *buff,u_char *server,  
NUTDEVICE *sostream);
```

-ใช้สำหรับติดต่อไปยังเมลล์เซฟเวอร์ที่ให้บริการSMTPเพื่อขอ ดู บริการที่มี

```
int SendMailFrom(TCPSOCKET *sock,u_char *buff,u_char  
*mailfrom,NUTDEVICE *sostream);
```

-สามารถส่งเมลล์จากตัว Therminal Devices ไปยังเครื่องปลายทางได้

```
int SendRcptTo(TCPSOCKET *sock,u_char *buff,u_char *rcpt,  
NUTDEVICE *sostream);
```

-ใช้สำหรับติดต่อไปยังเมลล์เซฟเวอร์ที่ให้บริการSMTPเพื่อส่งชื่อผู้รับ

```
int SendData(TCPSOCKET *sock,u_char *buff,u_char *text,  
NUTDEVICE *sostream);
```

-ใช้สำหรับติดต่อไปยังเมลล์เซฟเวอร์ที่ให้บริการSMTPเพื่อส่งข้อความของจดหมาย

```
int QuitMail(TCPSOCKET *sock,u_char *buff,NUTDEVICE *sostream);
```

-ใช้สำหรับติดต่อไปยังเมลเซิร์ฟเวอร์ที่ให้บริการSMTPเพื่อสิ้นสุดการทำงาน

สิ่งที่ต้องทดสอบ

- สามารถ Connect เข้าสู่ Smtп Server ได้
- สามารถส่งเมลจากที่ต่างๆเข้าสู่ Smtп Server ได้ทาง telnet port 25

ขั้นตอนการทดสอบ

-ทำการส่งเมลจาก การ telnet ไปยังที่ต่างๆ แล้วสามารถเรียกดูได้ โดยการเรียกดูจาก web หรือจากการ telnet

-ทำการรับเมลจาก การเตรียมข้อความที่จะส่งไว้ แล้วสามารถเรียกดูได้ โดยการเรียกดูจาก web หรือ จากการ telnet

ผลการทดสอบ

สามารถส่งเมลจากตัว Terminal Devices ไปยังเครื่องปลายทางที่เราระบุไว้ได้ โดยสามารถส่งได้ตาม protocol smtp โดยการเปิด port 25 หรือ ส่งไปโดยการที่เราเตรียมข้อความที่จะส่งไว้ได้

ข้อจำกัดของการทำงาน

- 1.เนื่องจากมีข้อจำกัดด้านหน่วยความจำที่ใช้งาน การรับเมลจึงไม่สามารถรับเมลที่มีขนาดใหญ่เกินกว่าขนาดของตัวแปรที่ได้จองไว้ (ในโปรแกรมระบุไว้ที่ 1024 ไบต์)
- 2.การทำงานของโปรแกรมยังไม่สามารถรองรับการใช้ของโปรแกรมรับส่งเมลอื่นๆได้ทั้งหมด ซึ่งอาจมีสาเหตุมาจากความล่าช้าของการรับส่งแพ็คเกจ หรือใช้รูปแบบการรับส่งที่ต่างกัน เช่น โปรแกรมเอทล์คเอ็กเพรส

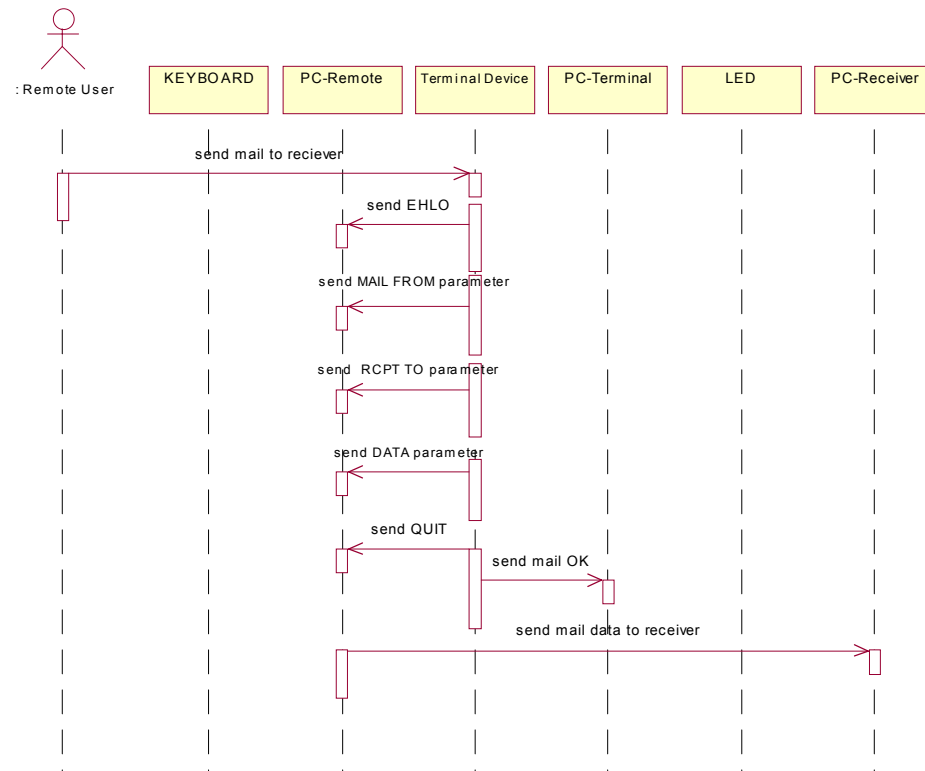
แนวทางในการใช้งานและพัฒนาต่อเนื่อง

- 1.ใช้ในงานที่ไม่ต้องการความปลอดภัยมากนักและมีการใช้งานที่ง่ายเนื่องจากงานผู้ใช้งานส่วนใหญ่สามารถรับส่งเมลได้อยู่แล้ว
- 2.สามารถเขียนโปรแกรมให้ความสามารถที่หลากหลายขึ้น เช่น มีรายนามผู้ใช้งานหลายคน(multiuser)

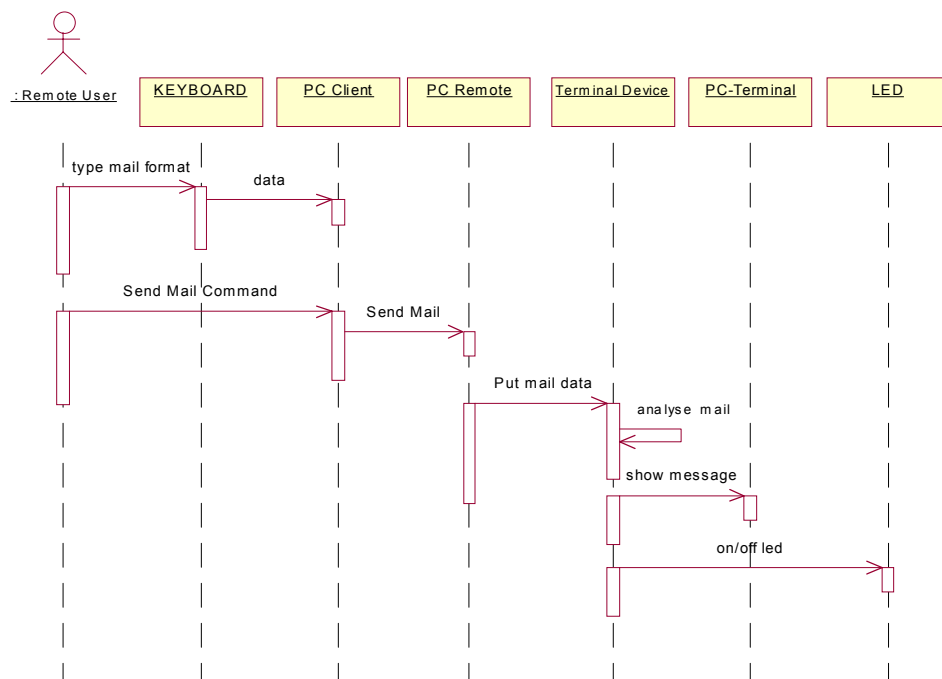
SMTP COMMANDS

คำสั่ง	รายละเอียด	คำสั่งที่ใช้
EHLO	ใช้เมื่อClientของอีเมลต้องการเริ่มติดต่อกับ Server	ใช่
MAIL	เริ่มเข้าสู่สถานะการส่ง E-mail	ใช่
RCPT	เป็นคำสั่งเพื่อระบุ E-mail ที่จะส่งที่ละฉบับโดยเป็นคำสั่งที่ใช้ต่อจาก MAIL	ใช่
DATA	จะเป็นคำสั่งที่ใช้ต่อจาก RCPT เพื่อส่งข้อมูลจาก Email	ใช่
SEND	ทำหน้าที่เหมือนคำสั่ง DATA แต่ไม่ค่อยมีผู้ใช้งาน	ไม่ใช่
SOML	ทำหน้าที่เหมือนคำสั่ง DATA แต่ไม่ค่อยมีผู้ใช้งาน	ไม่ใช่
SAML	ทำหน้าที่เหมือนคำสั่ง DATA แต่ไม่ค่อยมีผู้ใช้งาน	ไม่ใช่
VRFY	เป็นคำสั่งที่ใช้เพื่อตรวจสอบความถูกต้องของชื่อและเมลบ็อกซ์	ใช่
EXPN	เป็นคำสั่งที่ใช้เพื่อตรวจสอบรายละเอียดของลิสต์รายชื่อ	ไม่ใช่
HELP	เป็นคำสั่งที่ใช้เพื่อตรวจสอบคำสั่งที่สามารถใช้งานได้กับ Server	ไม่ใช่
NOOP	เป็นคำสั่ง No Operation เมื่อServerได้รับคำสั่งก็จะตอบ OK กลับมา	ใช่
QUIT	เป็นคำสั่งที่ใช้เพื่อบอกว่าสิ้นสุดการติดต่อ	ใช่
RSET	ยกเลิกการส่งข้อมูลในขณะนี้	ใช่
TURN	เป็นคำสั่งที่สลับหน้าที่ของผู้ส่งข้อมูลมารับข้อมูล	ไม่ใช่

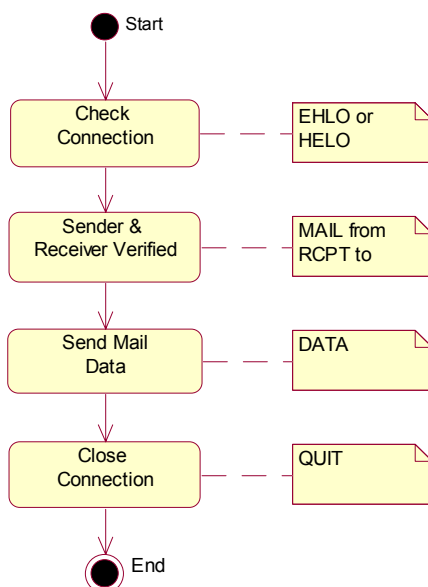
ตารางที่ 5-2 แสดง SMTP Commands ที่ได้เลือกใช้ในวิทยานิพนธ์



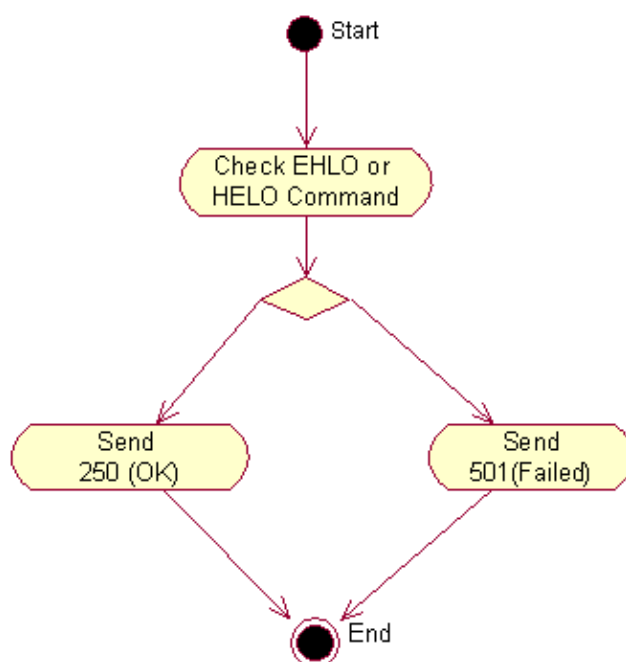
รูปที่ 5-10 แสดง Sequence Diagram ของการใช้งาน SMTP send



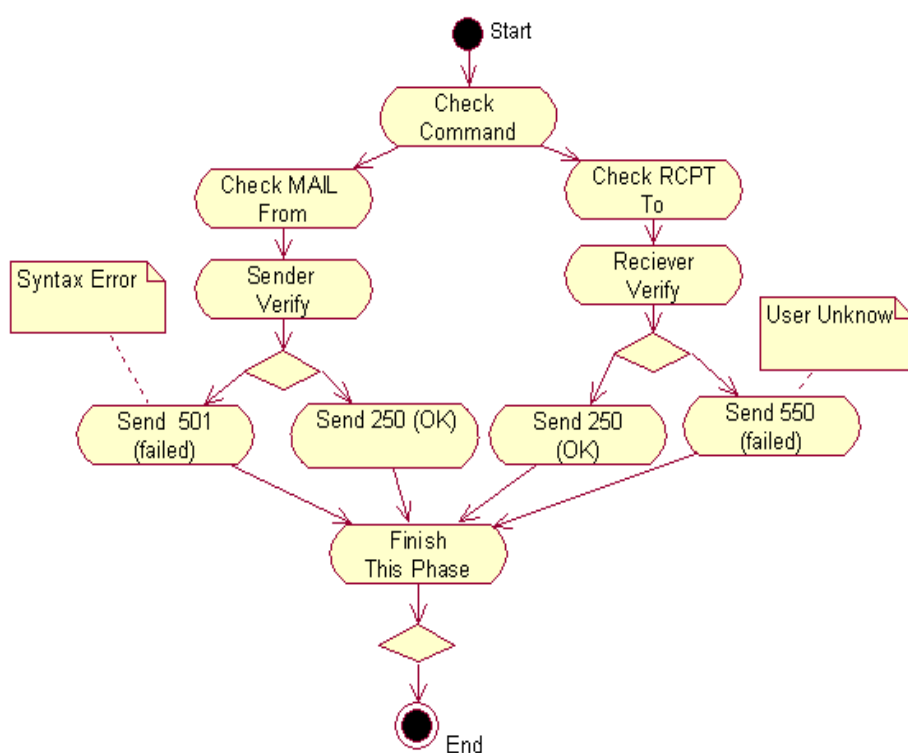
รูปที่ 5-11 แสดง Sequence Diagram ของการใช้งาน SMTP receive



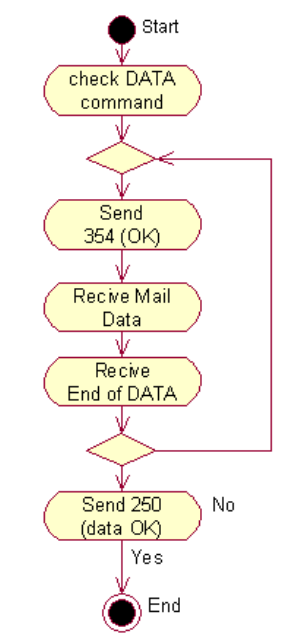
รูปที่ 5-12 แสดงสถานะการทำงานโปรโตคอลSMTPฝั่งรับ



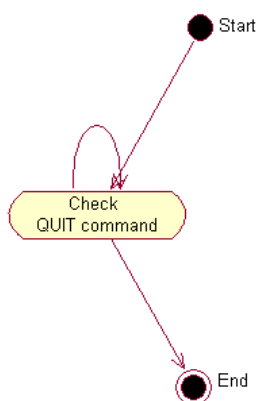
รูปที่ 5-13 แสดงขั้นตอนการตรวจสอบสถานะการเชื่อมต่อ



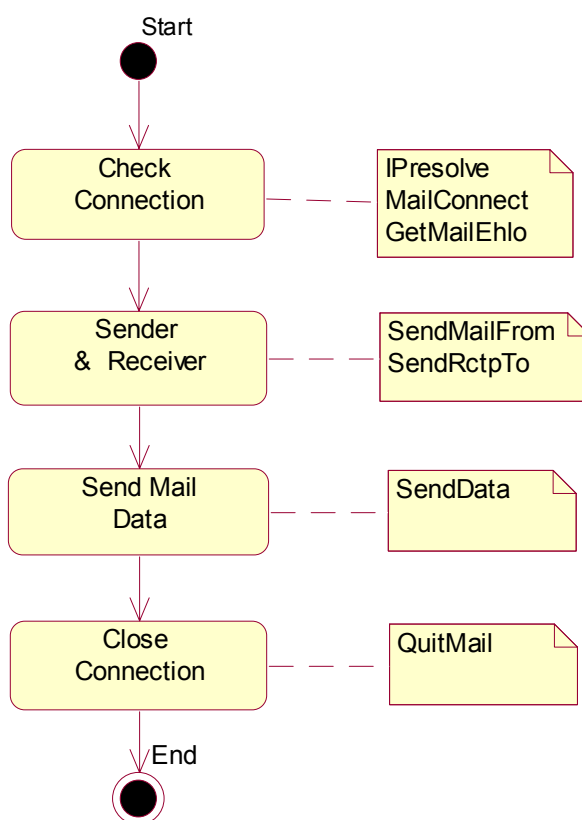
รูปที่ 5-14 แสดงขั้นตอนการตรวจสอบรายนามผู้ส่งและผู้รับ



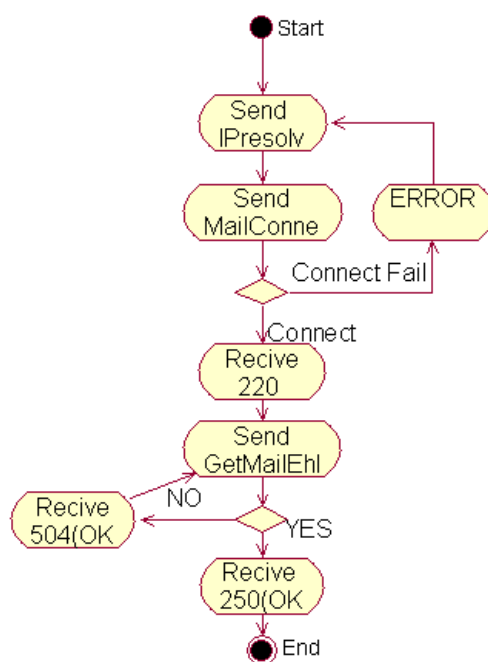
รูปที่ 5-15 แสดงขั้นตอนการส่งเนื้อหาของจดหมาย



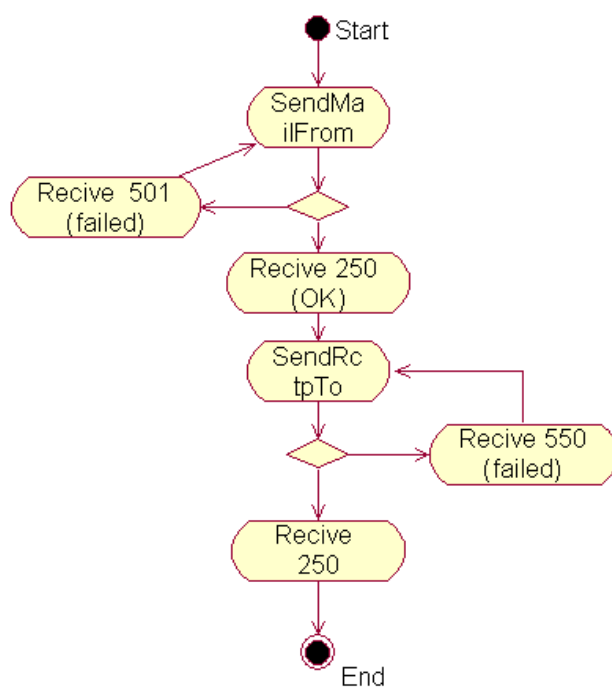
รูปที่ 5-16 แสดงขั้นตอนการจบการทำงาน



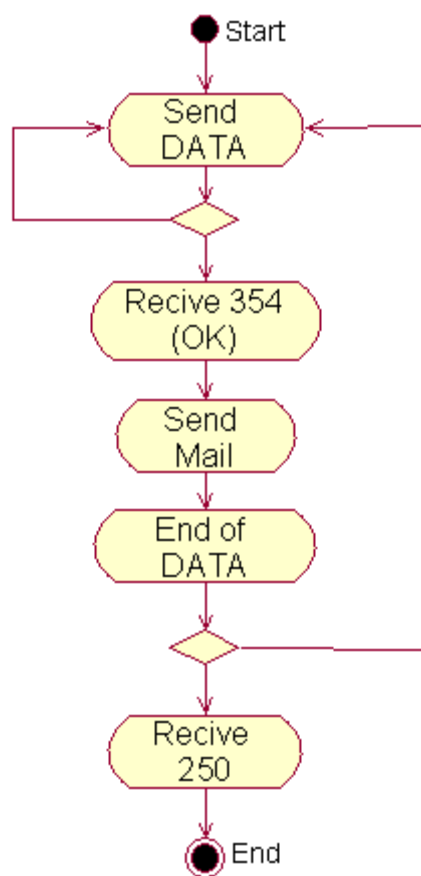
รูปที่ 5-17 แสดงสถานะการทำงานโปรโตคอลSMTPฝั่งส่ง



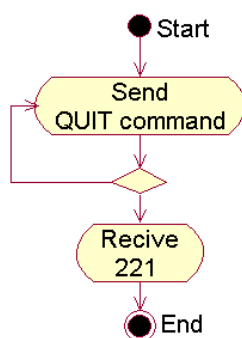
รูปที่ 5-18 แสดงขั้นตอนการตรวจสอบสถานะการเชื่อมต่อ



รูปที่ 5-19 แสดงขั้นตอนการตรวจสอบรายนามผู้ส่งและผู้รับ



รูปที่ 5-20 แสดงขั้นตอนการส่งข้อความของจดหมาย



รูปที่ 5-21 แสดงขั้นตอนการจบการทำงาน

SNMP

จุดประสงค์การใช้งานโปรโตคอล SNMP

การควบคุมอุปกรณ์หลายๆตัวพร้อมกันจำเป็นต้องมีมาตรฐานการควบคุมเดียวกัน แต่สำหรับอุปกรณ์Embedded นั้นมีข้อจำกัดหลายอย่างที่ไม่สามารถใช้มาตรฐานการควบคุมที่มีความซับซ้อนได้ การใช้โปรโตคอล SNMP ซึ่งดัดแปลงให้ใช้งานได้กับไมโครคอนโทรลเลอร์ขนาดเล็กได้จึงมีความเหมาะสมในการควบคุมระดับหนึ่ง

หลักการออกแบบ

การออกแบบโปรโตคอล SNMP สามารถแบ่งได้เป็นส่วนย่อยๆดังนี้

- 1.SNMP agent เป็นส่วนที่ทำการแยกแยะคำสั่งของSNMPว่าทางผู้ส่งมีสิทธิ์ในการเข้าถึงข้อมูลหรือไม่และกระทำต่อข้อมูลที่เก็บอยู่ในMIB
- 2.MIB เป็นส่วนของโครงสร้างที่เราออกแบบไว้เฉพาะสำหรับทำการควบคุม,อ่านค่า และเขียนค่าลงไปในการออกแบบต้องคำนึงว่าบอร์ดนั้นมีหน่วยความจำที่จำกัดจึงไม่สามารถที่จะใช้งานMIBที่สมบูรณ์ครบถ้วนได้จึงต้องมีการระบุถึงnodeที่จำเป็นต้องใช้จริงๆเท่านั้น
- 3.SNMP decode/encode ทำการencodeและdecode ข้อมูลเพื่อเก็thydityที่ที่ได้รับมาหรือเพื่อส่งกลับ

ฟังก์ชันการทำงาน

สามารถใช้โปรแกรมที่จัดการทางฝั่ง Management เข้ามาดูข้อมูลต่างๆได้ โดยมีคำสั่งการใช้งาน Snmp Agent ดังนี้

Get-request	ใช้สอบถามข้อมูลจากตัวเอเจนต์ที่อยู่บนอุปกรณ์ที่ต้องการตรวจสอบในระบบเครือข่าย
Get-next-request	ใช้สอบถามข้อมูลที่เรียงเป็นลำดับ เช่นข้อมูลที่เก็บอยู่ในรูปตาราง
Get-response	เอเจนต์ส่งคำตอบกลับมายังผู้สอบถาม
Set-request	ใช้เปลี่ยนแปลงค่าตัวแปรที่เอเจนต์รับผิดชอบอยู่
Trap	ใช้แจ้งเหตุการณ์ที่เกิดขึ้นในเครือข่าย เช่นการเริ่มต้นทำงานใหม่ของ อุปกรณ์ หรือเส้นทางขัดข้อง

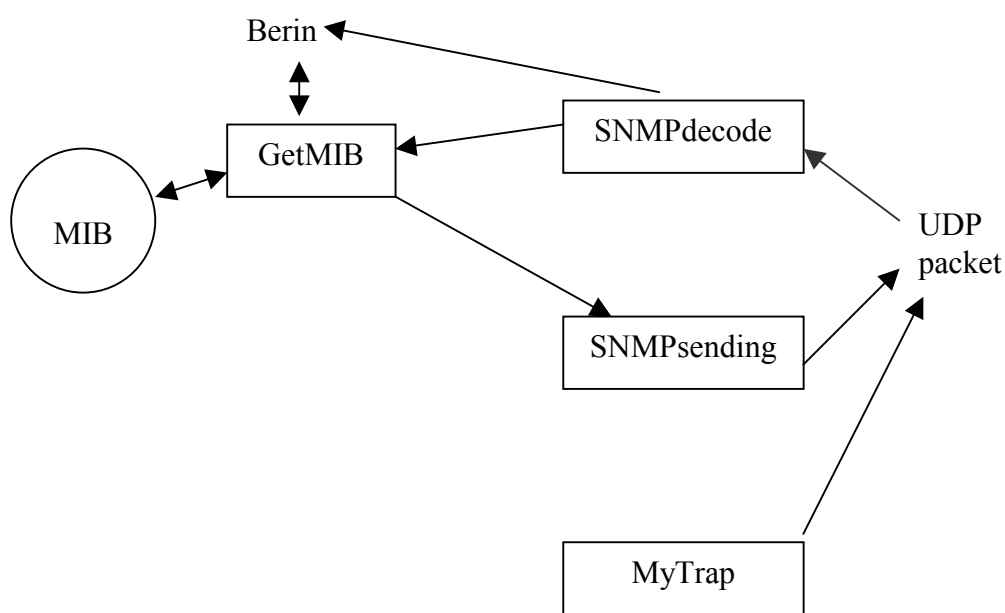
สิ่งที่ต้องทดสอบ

- สามารถรับการ Connect จากโปรแกรมที่ใช้จัดการได้ (SnmpManagement) โดยผ่านทาง port UDP 161 สำหรับ4 คำสั่งแรก (Get-request,Get-next-request,Get-response, Set-request)
- สามารถรับการ Connect จากโปรแกรมที่ใช้จัดการได้ (SnmpManagement) โดยผ่านทาง port UDP 162 สำหรับคำสั่งสุดท้าย (Trap)

ขั้นตอนการทดสอบ

- สามารถรับการ Connect จากโปรแกรมที่ใช้จัดการได้ (SnmpManagement)
- สามารถสอบถามข้อมูลจากตัวเอเจนต์ที่อยู่บนอุปกรณ์ที่ต้องการตรวจสอบในระบบเครือข่ายโดยการเรียกดูจาก SnmpManagement
- สามารถสอบถามข้อมูลที่เรียงเป็นลำดับ เช่นข้อมูลที่เก็บอยู่ในรูปตารางโดยการเรียกดูจาก SnmpManagement
- เอเจนต์สามารถส่งคำตอบกลับมายังผู้สอบถามโดยการเรียกดูจาก SnmpManagement
- สามารถเปลี่ยนแปลงค่าตัวแปรที่เอเจนต์รับผิดชอบอยู่ โดยการเรียกดูจาก SnmpManagement
- สามารถแจ้งเหตุการณ์ที่เกิดขึ้นในเครือข่าย เช่นการเริ่มต้นทำงานใหม่ของ อุปกรณ์ หรือเส้นทางขัดข้องโดยการเรียกดูจาก SnmpManagement

การทำงานของโปรแกรม SNMP Agent



รูปที่ 5-22 แสดงความสัมพันธ์ของฟังก์ชันการทำงาน

การทำงานของโปรแกรม SNMP Agent ประกอบไปด้วยฟังก์ชันดังต่อไปนี้

1. SNMP decode เป็นฟังก์ชันที่ทำหน้าที่

- 1.1) ดึงค่าต่างๆจากแพ็คเกจยูดีพีทีที่ได้รับมา
- 1.2) บันทึกค่าต่างๆที่ตีความได้ให้กับตัวแปรโครงสร้างBerin ซึ่งจะเก็บข้อมูลของ SNMP แพ็คเกจที่ส่งเข้ามา ได้แก่
 - ขนาดของแพ็คเกจ
 - เวอร์ชันของSNMPแพ็คเกจที่ได้รับ
 - รหัสที่ใช้ในการสื่อสาร(community string)
 - ชนิดของคำสั่งSNMPที่ทำการร้องขอ
 - รหัสของSNMPแพ็คเกจที่ได้รับ
 - ค่าความผิดพลาดของคำสั่งในแพ็คเกจ
 - ตำแหน่งที่ผิดพลาดของคำสั่งในแพ็คเกจ
 - Object IDที่ต้องการกระทำ
 - ค่าที่ใช้สำหรับObject IDนั้นๆ

2. SNMP sending เป็นฟังก์ชันที่ทำการเข้ารหัสSNMPแพ็คเกจที่จะทำการส่งกลับไปยังฝั่ง SNMP manager โดยจะส่งแพ็คเกจ SNMP ชนิดgetresponse ซึ่งเป็นแพ็คเกจชนิดที่เอเจนต์ทำการตอบกลับไปยังฝั่งSNMP Manager

3. GetMIBเป็นฟังก์ชันที่จะนำค่าชนิดของคำสั่งSNMPที่ทำการร้องขอมาประมวลผลเพื่อจะนำค่าที่ต้องการกลับไปให้ฝั่งที่ร้องขอ โดยมีรายละเอียดดังต่อไปนี้

3.1) ทำการตรวจสอบค่าชนิดของคำสั่งSNMPที่ทำการร้องขอจากตัวแปร โครงสร้าง Berin มาพิจารณาว่าเป็นคำสั่งชนิดใด

- getrequest ทำการร้องขอค่าของObject ID ที่ระบุ
- getnextrequest ทำการร้องขอค่าของObject IDที่อยู่ต่อจากค่าObject ID ที่ระบุ
- setrequest ทำการกำหนดค่าที่ได้ระบุมาให้กับObject IDนั้นๆ

3.2) ทำการตรวจสอบสิทธิในการเข้าถึงMIBของผู้ร้องขอโดยนำรหัสที่ใช้ในการสื่อสาร(community string)เปรียบเทียบกับรหัสผ่านที่ได้รับอนุญาตไว้โดยรหัสผ่านจะมีอยู่สองชนิดคือ รหัสผ่านที่ใช้สำหรับการอ่านค่าต่างๆไป เช่น คำสั่ง getrequest, getnextrequest กับรหัสผ่านที่ใช้สำหรับกำหนดค่าต่างๆให้Object IDที่ระบุโดยคำสั่งsetrequest

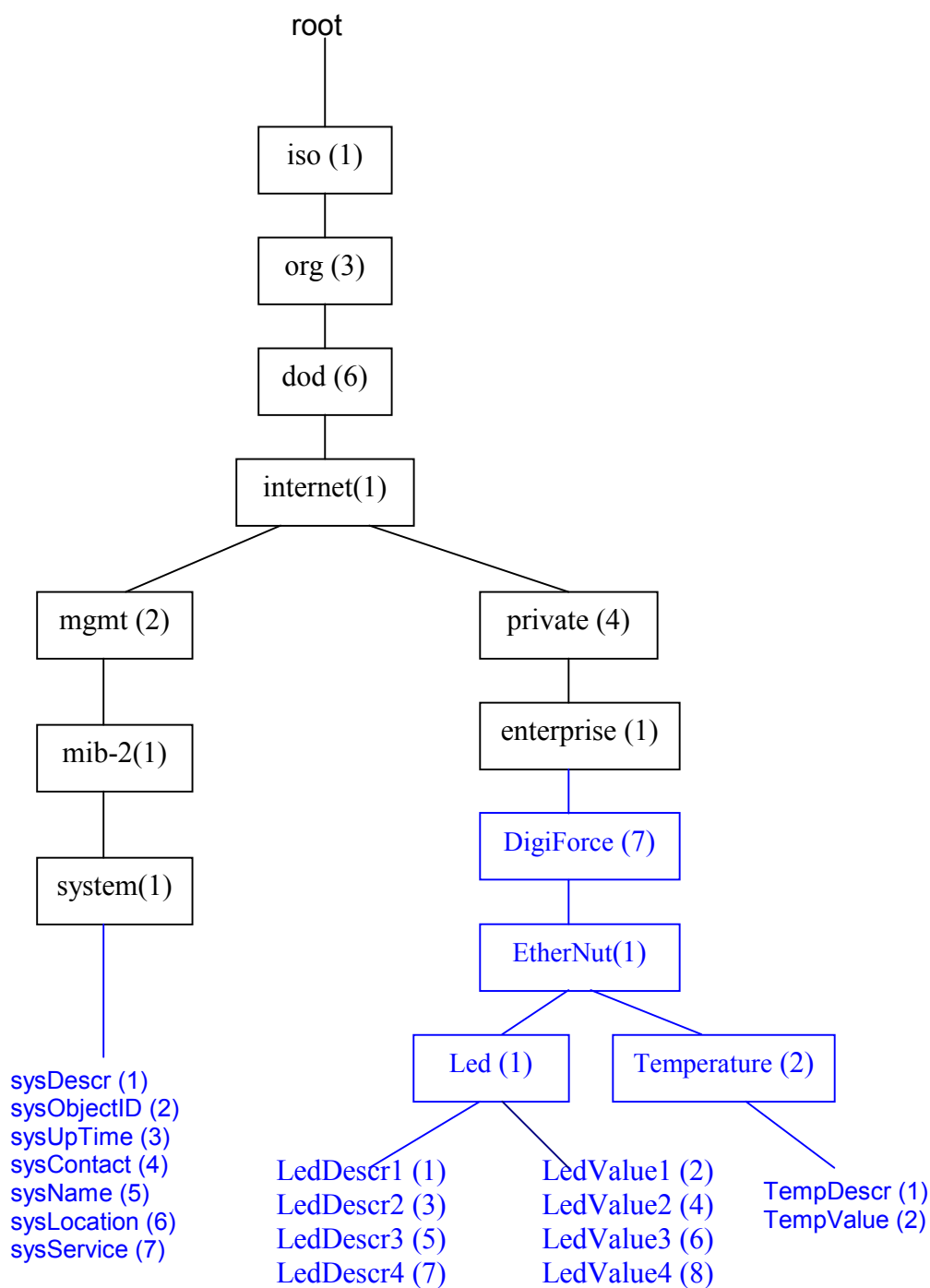
3.3) หากผู้ร้องขอมีสถานะสิทธิในการกระทำใดๆต่อObject IDนั้นๆก็จะกระทำการอ่านหรือกำหนดค่าให้กับ Object ID ที่ระบุ โดยค่าObject IDต่างๆจะถูกกำหนดให้เป็นตัวแปรในลักษณะของMIB

3.4) นำค่าที่ได้จากการกระทำในข้อ3.3มาเรียบเรียงให้อยู่ในรูปแบบของSNMPแพ็คเกจก่อนส่งให้ฟังก์ชันSNMP Sendingทำการตรวจสอบแพ็คเกจและทำการส่งกลับให้กับฝั่งManager

4.MyTrap เป็นเทรด (THREAD)ที่ทำงานแยกออกมาจากฟังก์ชันอื่นๆเพราะต้องใช้พอร์ตในการสื่อสารต่างกันและในการทำงานนั้นต้องทำการตรวจสอบแยกต่างหากกับกระบวนการอื่นๆโดยฟังก์ชันนี้จะทำการส่งSNMPแพ็คเกจชนิดคำสั่งTrap กลับไปให้กลับฝั่งManager โดยขึ้นอยู่กับเงื่อนไขที่ระบุให้กับ SNMP เอเจนท์ให้ทำการส่ง เช่น ให้ส่งtrap ทุกครั้งที่ทำการรีบูทระบบ

การออกแบบ MIB

MIB ที่ได้ทำการกำหนดไว้มีรายละเอียดดังต่อไปนี้



รูปที่ 5-23 แสดงโครงสร้างของMIBที่ทำการออกแบบให้กับชิ้นงาน

เนื่องจากในอุปกรณ์ปลายทางขนาดเล็กนั้นมีข้อจำกัดในการทำงานอยู่หลายส่วน เช่น ทางด้านความเร็วของตัวไมโครคอนโทรลเลอร์ที่ช้ากว่าซีพียูของคอมพิวเตอร์ทั่วไปมาก, หน่วยความจำที่จำกัด, ไม่มีหน่วยที่ใช้เก็บข้อมูลถาวรมากนัก เป็นต้น ดังนั้นการออกแบบMIBให้กับอุปกรณ์ปลายทางขนาดเล็กจึงต้องจำกัดขนาดและการใช้งานให้อยู่ในความต้องการใช้งานพื้นฐานเท่านั้น โดยมีหลักการออกแบบดังนี้

1. โหนดที่ใช้งานจะต้องเป็นโหนดที่พื้นฐานที่สุดและมีการเปลี่ยนแปลงที่ง่ายต่อการใช้งาน
2. จะไม่กำหนดโหนดที่เกี่ยวข้องกับสถิติการทำงาน เช่น โหนดที่เก็บจำนวนแพ็กเก็ตUDPที่เสียหายซึ่งไม่มีความจำเป็นกับการใช้งานมากนัก
3. โหนดที่กำหนดขึ้นใช้งานเองโดยไม่มีอยู่ในMIBมาตรฐาน จะกำหนดไว้ต่อจากโหนด enterprise เพราะเป็นโหนดที่องค์กรนั้นได้กำหนดขึ้นเอง จึงไม่ถือเป็นมาตรฐาน
4. โหนดที่กำหนดขึ้นเองนั้นจะไม่สามารถใช้คำสั่งgetnextในการอ่านค่าได้ เพราะทางคณะผู้จัดทำคำนึงถึงความปลอดภัยในการใช้งาน เนื่องจากโหนดที่กำหนดขึ้นเองนั้นเพื่อใช้ในการควบคุมตัวอุปกรณ์ ดังนั้นหากสามารถใช้คำสั่งgetnextได้ก็จะทำให้ทราบค่าต่างๆที่ใช้ในการควบคุมทั้งหมดได้ รายละเอียดของแต่ละโหนดที่ได้ทำการกำหนดไว้ใช้งานมีรายละเอียดดังต่อไปนี้

โหนดมาตรฐาน

OID (หมายเลขโหนด)	คำจำกัดความ	รายละเอียด	ชนิดของโหนด	โหมด
1.3.6.1.2.1.1.1.0	sysDescr	ชื่อและคำอธิบายของระบบ	string	Read Only
1.3.6.1.2.1.1.2.0	sysObjectID	ระบุIDของผู้ผลิต(ในที่นี้กำหนดให้เป็นNULL)	object	Read Only
1.3.6.1.2.1.1.3.0	sysUpTime	ระบุเวลาทั้งสิ้นตั้งแต่เริ่มทำงานถึงปัจจุบัน(ยังไม่รองรับการทำงานเนื่องจากยังไม่มีระบบ (RTC))	timeticks	Read Only
1.3.6.1.2.1.1.4.0	sysContact	ชื่อและที่อยู่ของผู้ควบคุมระบบ	string	Read Write
1.3.6.1.2.1.1.5.0	sysName	ชื่อที่ผู้ควบคุมกำหนดให้กับโหนด	string	Read Write
1.3.6.1.2.1.1.6.0	sysLocation	ชื่อของสถานที่ซึ่งระบบติดตั้งอยู่	string	Read Write
1.3.6.1.2.1.1.7.0	sysService	จำนวนโหนดที่ให้บริการ (ในที่นี้กำหนดเป็นNULL)	string	Read Only

หมายเหตุ โหนดที่กำหนดค่าเป็นNULLเนื่องจากไม่สามารถกำหนดค่ามาตรฐานให้ได้ ส่วนโหนดที่ยังทำงานไม่ได้เป็นการกำหนดไว้เพื่อใช้งานต่อไปในอนาคต

ตารางที่ 5-3 แสดงโหนดมาตรฐานที่ใช้ใน SNMP

โหนดที่กำหนดเพิ่มเติม

OID (หมายเลขโหนด)	คำจำกัดความ	รายละเอียด	ชนิดของโหนด	โหมด
1.3.6.1.4.1.7.1.1.1.0	LedDescr1	รายละเอียดของLedหลอดที่1	string	Read write
1.3.6.1.4.1.7.1.1.2.0	LedValue1	ค่าที่กำหนดให้กับLedหลอดที่1	Int	Read write
1.3.6.1.4.1.7.1.1.3.0	LedDescr2	รายละเอียดของLedหลอดที่2	string	Read write
1.3.6.1.4.1.7.1.1.4.0	LedValue2	ค่าที่กำหนดให้กับLedหลอดที่2	Int	Read Write
1.3.6.1.4.1.7.1.1.5.0	LedDescr3	รายละเอียดของLedหลอดที่3	String	Read Write
1.3.6.1.4.1.7.1.1.6.0	LedValue3	ค่าที่กำหนดให้กับLedหลอดที่3	Int	Read Write
1.3.6.1.4.1.7.1.1.7.0	LedDescr4	รายละเอียดของLedหลอดที่4	String	Read write
1.3.6.1.4.1.7.1.1.8.0	LedValue4	ค่าที่กำหนดให้กับLedหลอดที่4	int	Read write
1.3.6.1.4.1.7.1.2.1.0	TempDescr	รายละเอียดของเซนเซอร์วัดอุณหภูมิที่1	string	Read write
1.3.6.1.4.1.7.1.2.1.0	TempValue	ค่าที่อ่านจากเซนเซอร์วัดอุณหภูมิที่1	int	Read Only

หมายเหตุ การกำหนดค่าให้กับหลอดLedนั้นจะกำหนดเป็นชนิด integer โดยหากมีค่ามากกว่า0ให้ทำการเปิดหลอดLed ,หากมีค่าเท่ากับ0จะทำการปิดหลอดLed

ตารางที่ 5-4 แสดงโหนดที่กำหนดเพิ่มเติม

ข้อจำกัดของการทำงาน

- 1.การทำงานยังไม่สามารถรับส่งที่หลายๆObject IDได้เพราะต้องทำการencodeเพิ่มเมื่อขนาดของแพ็คเกจมากกว่า127ไบต์
- 2.เนื่องจากMIBที่ใช้จำเป็นต้องมีขนาดเล็ก โหนดที่มีจึงเป็นเพียงบางส่วนเท่านั้น จึงอาจมีปัญหากับการใช้งานกับฝั่งจัดการบางตัว
- 3.เนื่องจากการใช้งานในการควบคุมนั้นจำเป็นต้องมีการดัดแปลงMIBเพิ่มเติมจากที่ได้ระบุไว้เดิม

แนวทางในการใช้งานและพัฒนาต่อเนื่อง

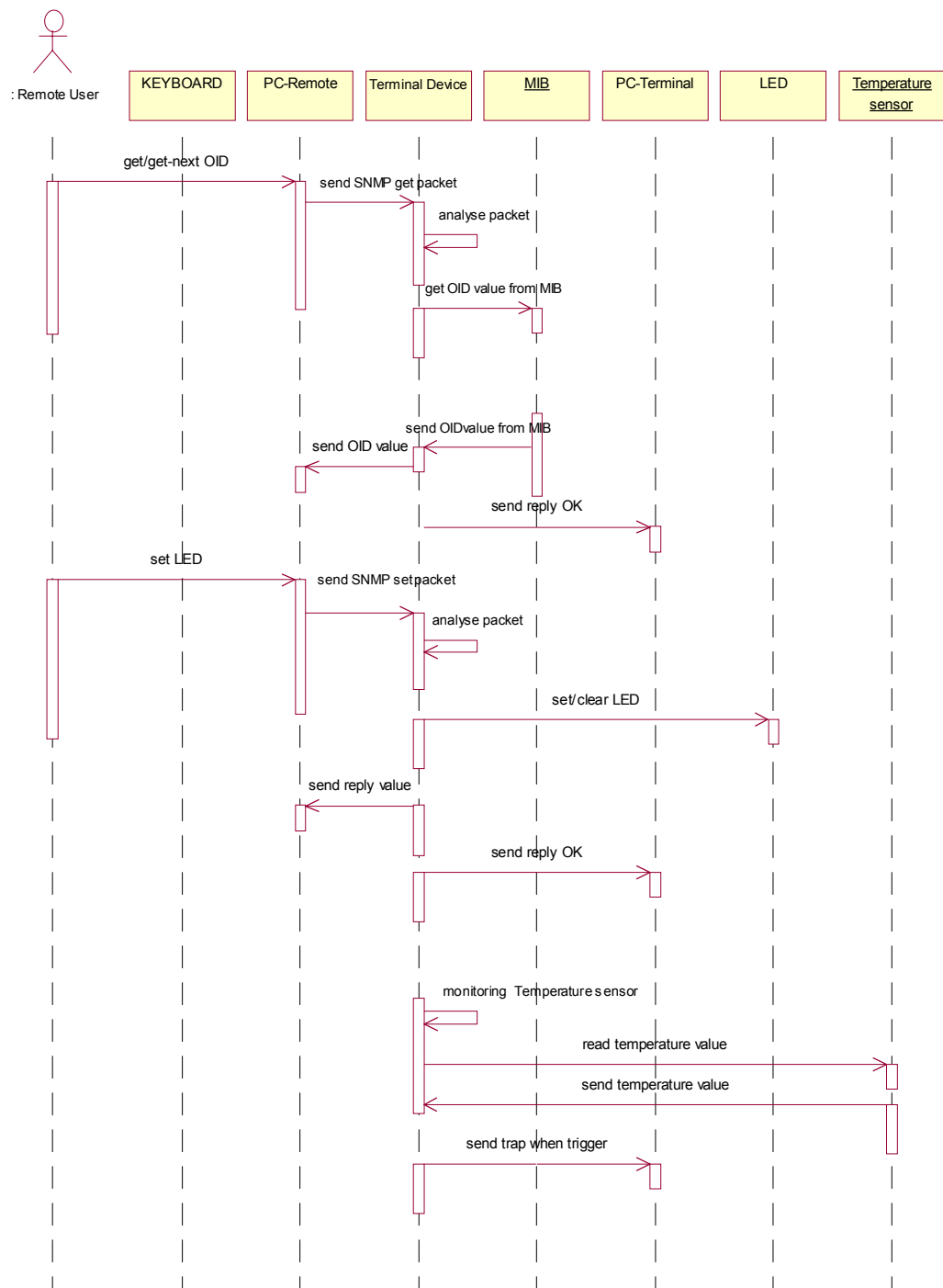
- 1.สร้างและกำหนดMIBให้มีมาตรฐานมากขึ้น
- 2.สามารถเขียนโปรแกรมให้ความสามารถที่หลากหลายขึ้น เช่น มีรายนามผู้ใช้งานหลายคน

(multiuser)

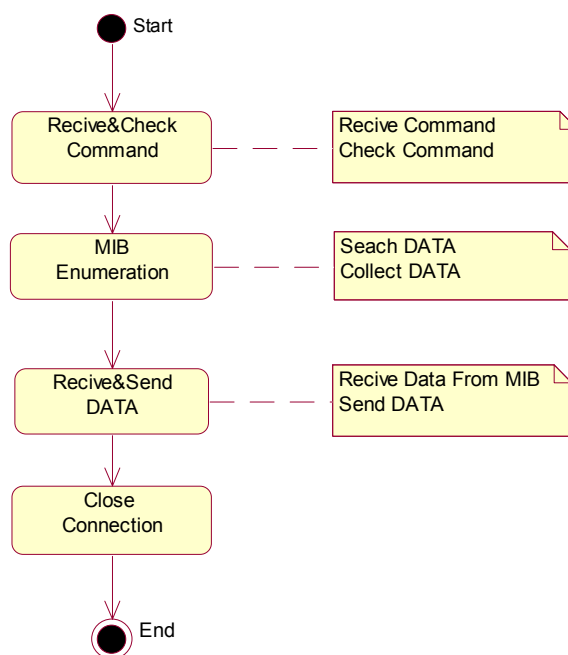
SNMP COMMANDS

ต1 5-5	คำสั่ง	รายละเอียด	คำสั่งที่ใช้	รางที่
	Get-request	ใช้สอบถามข้อมูลจากตัวเอเจนต์ที่อยู่บนอุปกรณ์ที่ต้องการตรวจสอบในระบบเครือข่าย	ใช่	
	Get-next-request	ใช้สอบถามข้อมูลที่เรียงเป็นลำดับ เช่นข้อมูลที่เก็บอยู่ในรูปตาราง	ใช่	
	Get-response	เอเจนต์ส่งคำตอบกลับมายังผู้สอบถาม	ใช่	
	Set-request	ใช้เปลี่ยนแปลงค่าตัวแปรที่เอเจนต์รับผิดชอบอยู่	ใช่	
	Trap	ใช้แจ้งเหตุการณ์ที่เกิดขึ้นในเครือข่าย เช่นการเริ่มต้นทำงานใหม่ของ อุปกรณ์ หรือเส้นทางขัดข้อง	ใช่	
	GetBulkRequest	ใช้สอบถามข้อมูลจากตัวเอเจนต์ที่อยู่บนอุปกรณ์ในระบบเครือข่าย	ไม่ใช่	
	Report	ใช้สอบถามข้อมูลจากตัวเอเจนต์โดยสามารถเข้าถึงข้อมูลได้หลายตัวพร้อมกัน	ไม่ใช่	

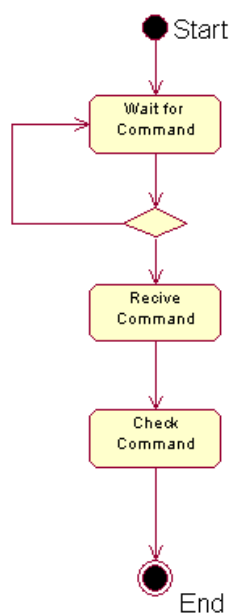
แสดง SNMP Commands ที่ได้เลือกใช้ในวิทยานิพนธ์



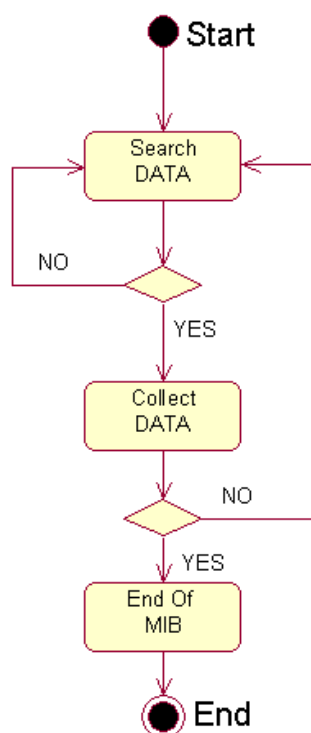
รูปที่ 5-24 แสดง Sequence Diagram ของการใช้งาน SNMP



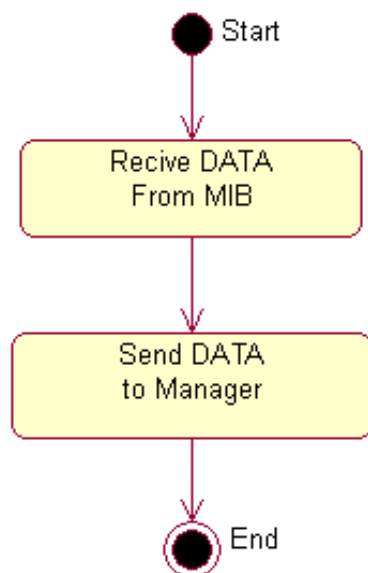
รูปที่ 5-25 แสดงสถานะการทำงานโปรแกรม SNMP agent



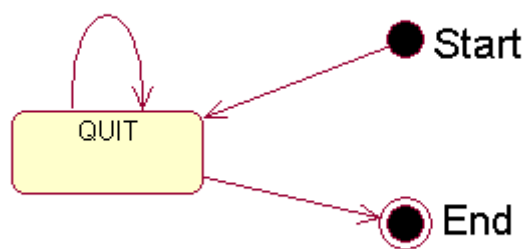
รูปที่ 5-26 แสดงขั้นตอนการรอรับและวิเคราะห์คำสั่งจากฝั่งจัดการ



รูปที่ 5-27 แสดงขั้นตอนการหาค่าจากMIB



รูปที่ 5-28 แสดงขั้นตอนการจัดการส่งค่าที่ได้จากMIBและส่งค่ากลับ



รูปที่ 5-29 แสดงขั้นตอนการจบการทำงาน